OAP200V

# Object-oriented analysis, design and programming

## 05 – Software project management with Maven

# Maven

- Introduction
- Using Maven (I)
  - Installing the Maven plugin for Eclipse
  - Creating a Maven Project
  - Building the Project
- Understanding the POM
  - Maven Lifecycle
  - Plugins
  - Repositories
- Using Maven (II)
  - A Project example

# Introduction to Maven

- Maven is a tool for software project management.
  - It provides the developer a complete build lifecycle framework
  - Based on a model of the project (POM = project object model)
  - Maven can manage a project's build, reporting and documentation from a central piece of information
- Executing Maven commands
  - look for POM file in Maven
  - run the command on the resources described in the POM

# Maven

- Handles following activities of a developer
  - Build
  - Documentation
  - Reporting
  - Dependencies
  - SCMs (Software Configuration Management)
  - Releases
  - Distribution
  - Mailing list

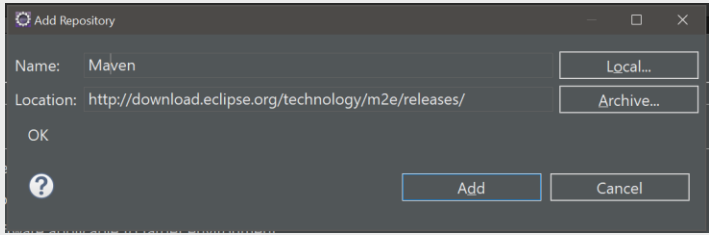
# Introduction to Maven

- It provides an extensible mean to build the software. Using additional plugins, the build process can include execution of tests, generation of documentation and deploy of application in different servers.
- Maven includes support for dependency management. It can download the required libraries and help to developers in the detection of libraries conflicts.

# Using Maven

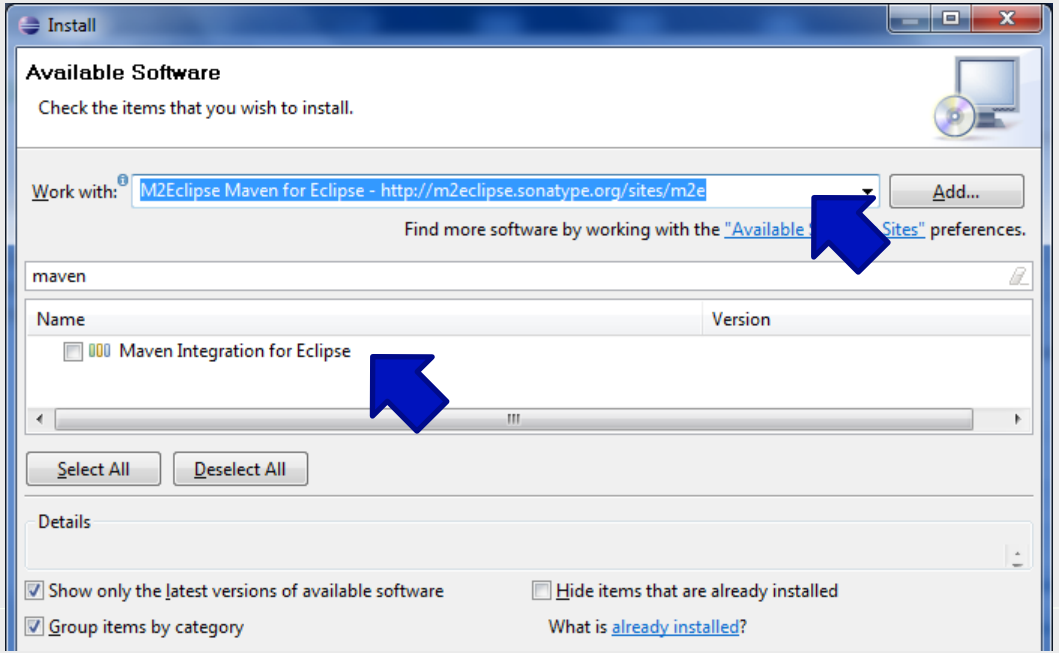
# Installing Maven plugin for Eclipse

- Many Eclipse downloads have M2Eclipse included already
  - Check if it is available in Window --> Preferences (Eclipse --> Preferences on Mac)
- Add the software update sites into the Eclipse
- Help --> Install New Software --> Add
  - Name: Maven
  - Location: https://download.eclipse.org/technology/m2e/releases/latest/
- Or: Help -> Eclipse Marketplace
  - Search by Maven
  - Click "Install" button at "Maven support in Eclipse IDE latest" section



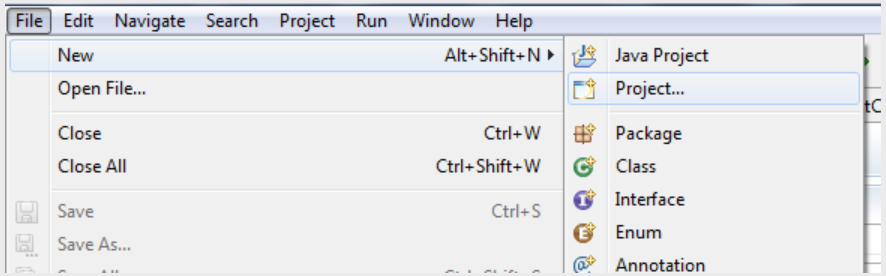
# Installing Maven plugin for Eclipse

- Search and install the plugin



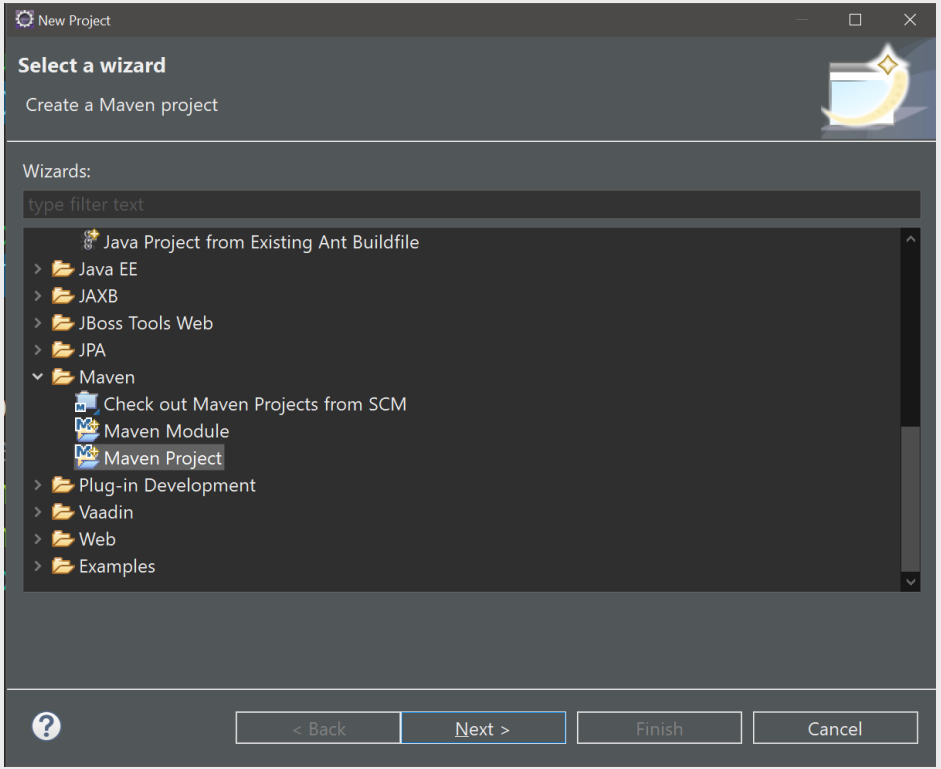
# Creating a Project

- Creating a project (using an archetype)
  - Select the option “New” -> “Project…” from the menu “File”



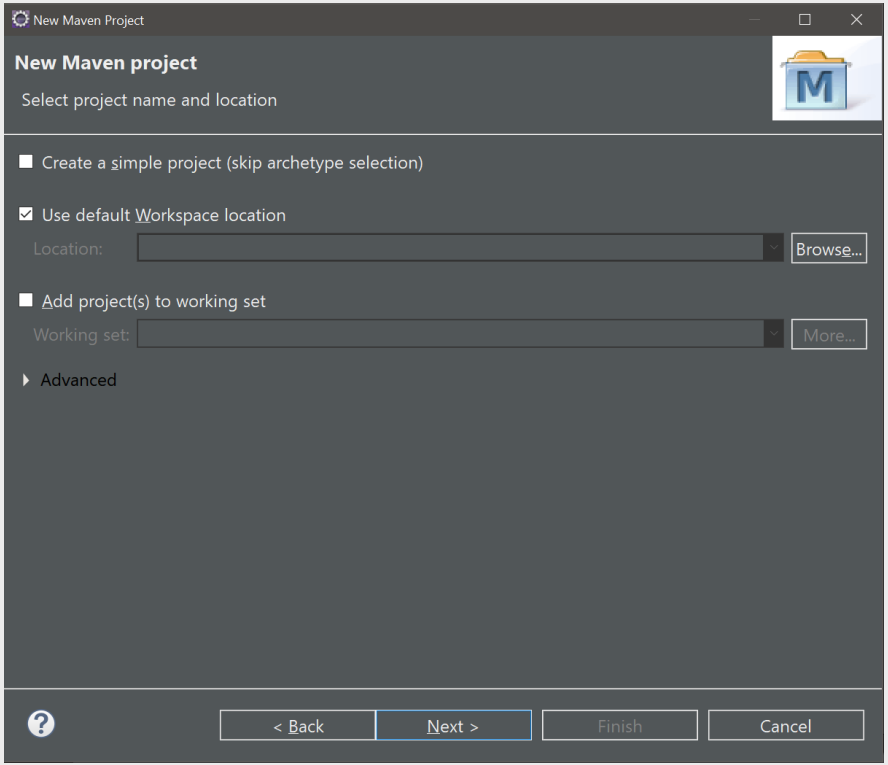
# Creating a Project

- Select the “Maven Project” type



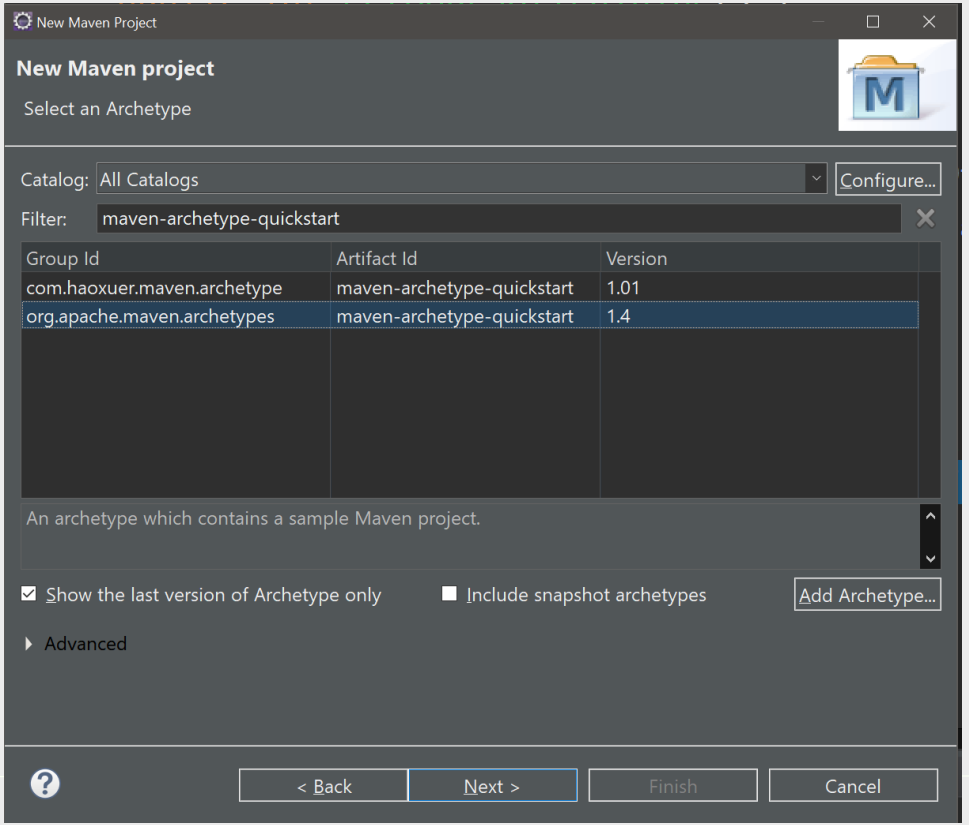
# Creating a Project

- If it is desired, change the default location and the working set for the new project



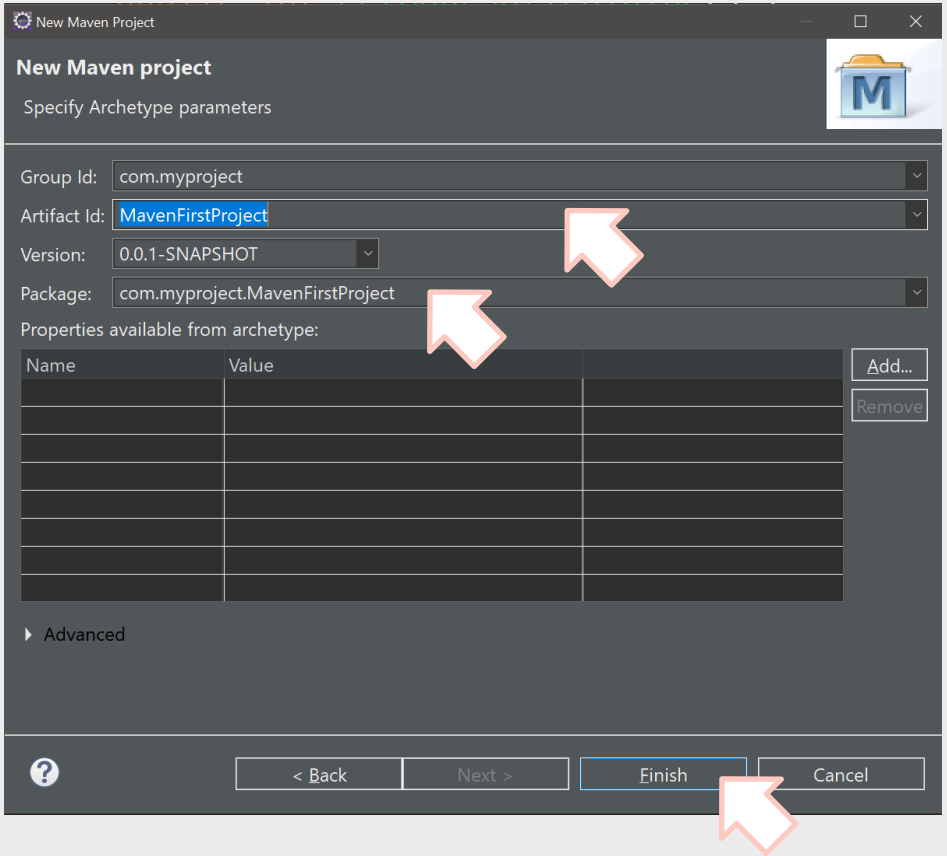
# Creating a Project

- Select the desired archetype.
  - e.g. maven-archetype-quickstart



# Creating a Project

- Define the project parameters



# Creating a Project

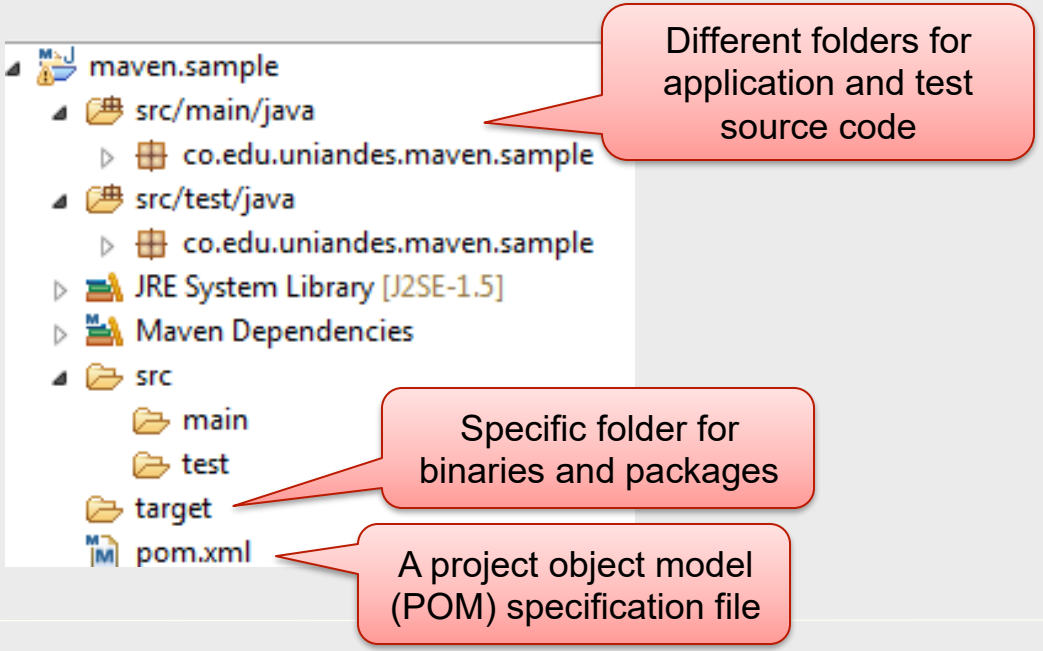
- Project Parameters:
  - **Group Id :** Name of the group of projects. Can be related to the company or application name. Ideally, must be unique (worldwide).
  - **Artifact Id :** Name of the component or application (project)
  - **Version :** Identifier for the version of the component. You can define different versions for the same component.
  - **Package Name :** Name of the java package where the initial source code will be generated.

# Reviewing the Project

- Reviewing (Modifying) the project
  - Usually, the archetype is only a skeleton that you can use as foundation for your projects.
  - It can provide source code and files in a maven-standard structure

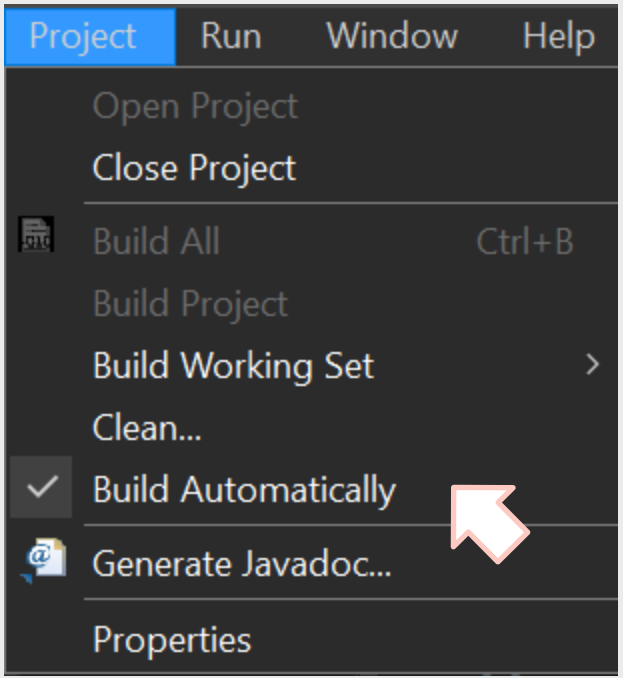
# Reviewing the Project

- Maven-standard structure



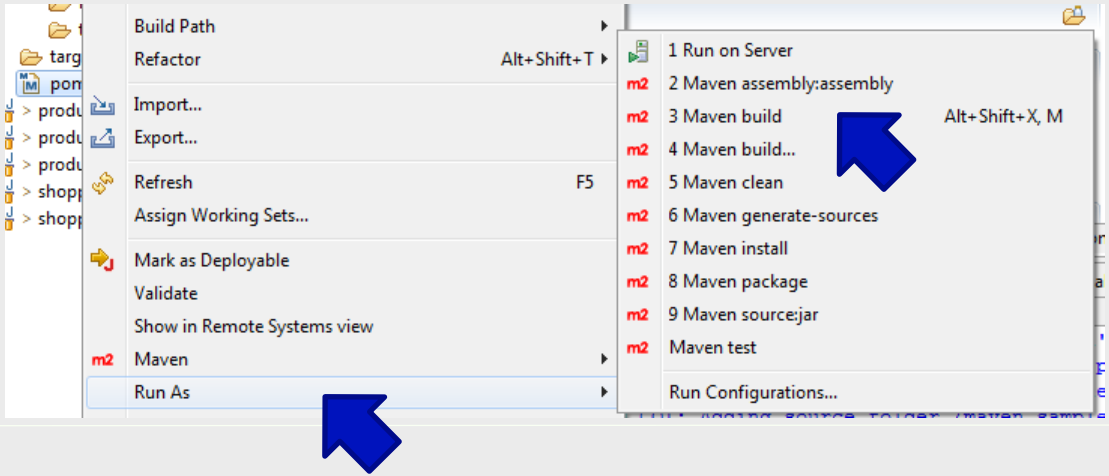
# Building the Project

- Building the project
  - If you are using the automatic build, the entire project is built after any file modification.



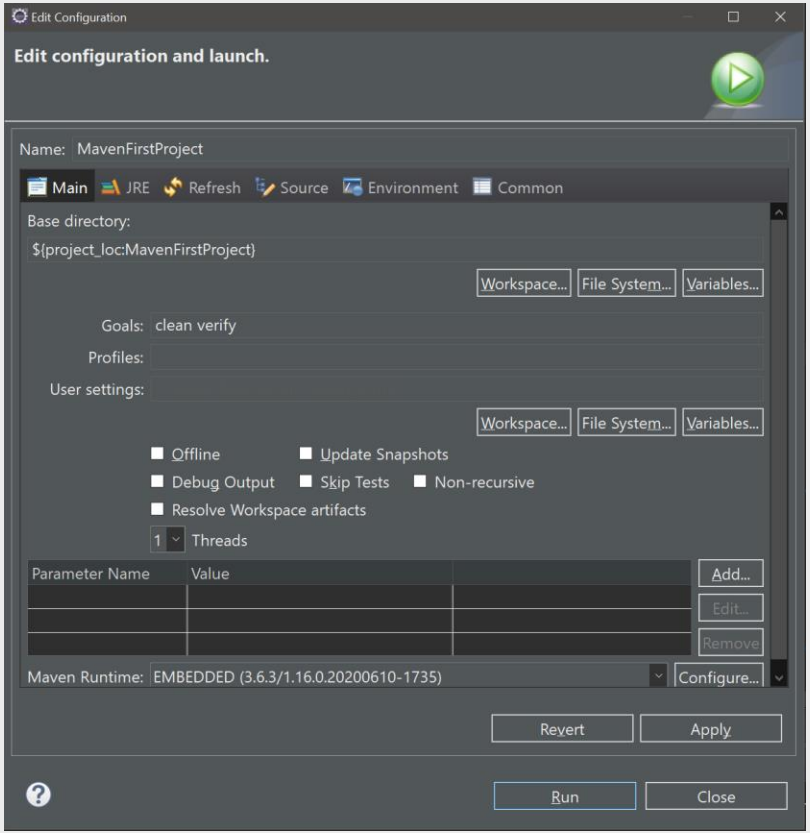
# Building the Project

- Building the project
  - Run As → Maven build
  - However, additional build types can be performed using Maven-specific menu options



# Run the build

- Enter `clean verify` in the Goals: field



# Understanding the POM

# Understanding POM

- Project Object Model
- XML file which holds the information about the project and configuration details used to build a project by Maven

# Understanding the POM

App.java | MavenFirstProject/pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.myproject</groupId>
  <artifactId>MavenFirstProject</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <name>MavenFirstProject</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>1.7</maven.compiler.source>
    <maven.compiler.target>1.7</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

Project properties

Dependencies

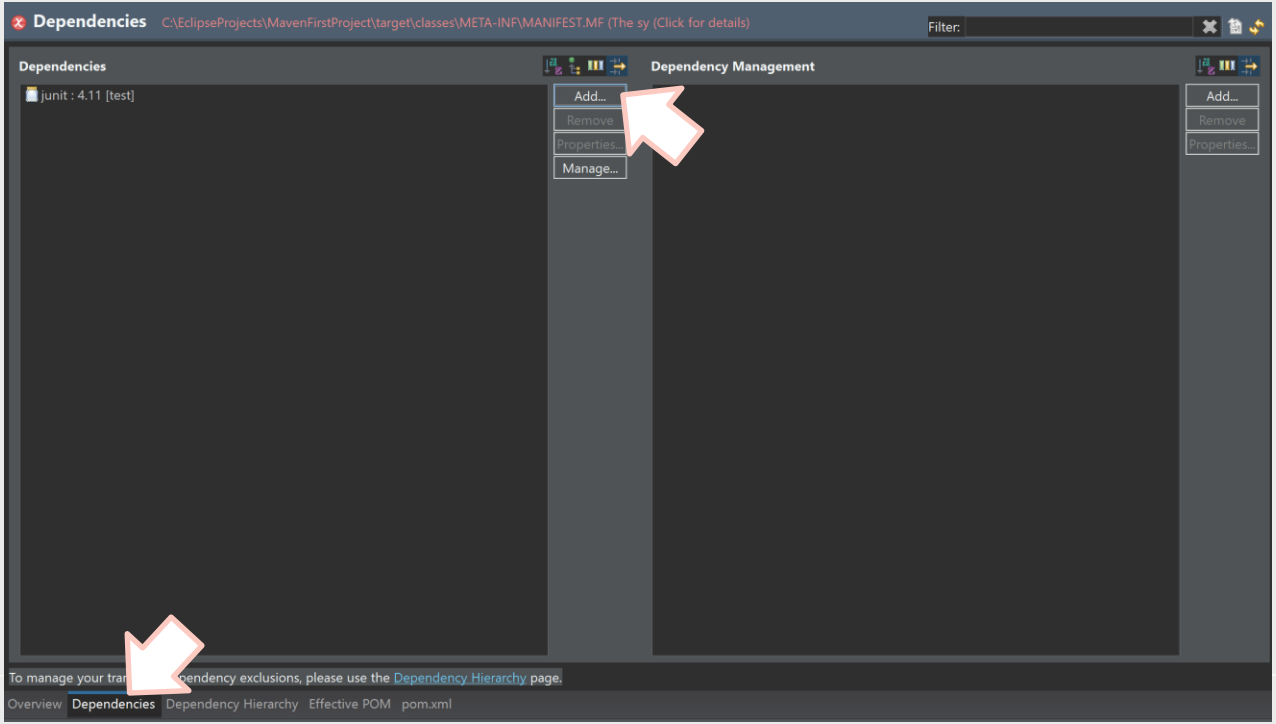
# Adding dependencies to your project

# Adding dependencies to the classpath of the project

- Software projects rarely work in isolation.
- In most cases, a project relies on reusable functionality in the form of libraries or is broken up into individual components to compose a modularized system.
- Dependency management is a technique for declaring, resolving and using dependencies required by the project in an automated fashion.

# Adding dependencies to the classpath of the project

- Select pom.xml
- Choose Dependencies tab
- Click Add
- Or copy xml code snippet from mvnrepository.com for the package you need



# Questions?